PRIYANSHU NEGI

Python Full Stack Developer | Django & React.js

+91 7428432234 | negips11april@gmail.com | linkedin.com/in/priyanshu-negi-b577b536b | github.com/Priyanshugitnegi | priyanshunegi-portfolio.vercel.app

PROFESSIONAL SUMMARY

Python full stack developer who has built and deployed production applications with Django, Django REST Framework and React.js, owning features from database design and REST APIs through to frontend integration and deployment. Experienced in query optimization across PostgreSQL and MySQL, and in integrating LLM APIs and retrieval-augmented generation (ChromaDB) into Django applications to build AI-powered features.

TECHNICAL SKILLS

Backend: Python (OOP, scripting, exception handling), Django, Django REST Framework, REST API design, token-based authentication, Django ORM, Django Admin, Django unit testing

Databases: PostgreSQL, MySQL, SQLite; relational modeling, schema design, complex queries, query optimization; ChromaDB (vector database)

AI & LLM: LLM API integration (Groq), Retrieval-Augmented Generation (RAG), semantic search, AI-assisted resume screening, Pydantic (structured output validation), PyMuPDF, python-docx

Frontend: React.js, JavaScript, HTML5, CSS3, Streamlit

Tools & Deployment: Git, GitHub, Postman, VS Code, Render, Vercel, Neon

PROJECTS

BioLogist: E-Commerce Platform | Django, Django REST Framework, MySQL, React.js
Live: biologistinfo.com | Backend: GitHub | Frontend: GitHub

- Built and deployed for a client: a live e-commerce platform with a product catalog, variants, shopping cart, and bulk product import.
- Designed relational database models for products, categories, subcategories, and product variants.
- Optimized inefficient product queries with `select_related()` and `prefetch_related()`, cutting query count per page by 40-50%.
- Reduced repeated load on the product-list endpoint with 10-minute response caching, cutting average response time by about 50%.
- Implemented variant availability checks at the database level using `Exists()` and `OuterRef()`.
- Built Excel-based bulk product import supporting 500+ products per upload, replacing hours of manual entry.
- Fixed duplicate cart entries by checking for an existing CartItem and updating its quantity instead of creating a new record.
- Resolved a catalog-number field-size limitation by increasing its capacity to 255 characters.
- Structured frontend-backend data handling with React state/effects and a dedicated API service layer.

TalentLens: AI Resume Screening System | Django, Django REST Framework, React.js, Groq API
Live: talent-lens-one.vercel.app | GitHub: TalentLens

- Built for a client: an AI-powered tool that screens multiple resumes against a job description and scores candidates, deployed live.
- Built Django REST Framework APIs that accept multiple PDF and DOCX resumes and screen them against a job description.
- Parsed resume text with PyMuPDF and python-docx, then extracted skills and computed matched and missing skills per candidate.
- Integrated the Groq LLM API to generate candidate assessments (strengths, additional skills, match score), using Pydantic models to validate structured responses.
- Stored screening history so past results can be reopened, with SQLite for local development and PostgreSQL configured for deployment.
- Implemented CSV and PDF export of screening reports, plus side-by-side candidate comparison.
- Built the React/Vite frontend to upload resumes, display match scores and strengths, and browse screening history.

AI Support Operations Copilot | Django REST Framework, Groq API, ChromaDB, Streamlit
Live: Dashboard | API | GitHub

- Built for a client: an AI-powered support operations platform that helps support teams manage tickets, analyze customer issues, and draft replies, deployed live.
- Built Django REST Framework APIs for support ticket management with token-based authentication (register, login, logout) and user-scoped querysets so each user can only access their own tickets and analyses.
- Designed ticket endpoints for create, list, and PATCH updates of status and priority, plus custom analyze and generate-reply actions and an analysis-history API.
- Integrated the Groq LLM API to summarize issues, classify category and priority, recommend agent actions, and draft customer replies, validating structured outputs with Pydantic.
- Implemented retrieval-augmented generation with ChromaDB, running semantic search over support knowledge articles managed through Django Admin to supply context to AI workflows.
- Built a Streamlit dashboard that calls the API over token-authenticated HTTP to create tickets, track metrics and recent activity, and review AI analyses.
- Wrote 9 Django tests; deployed the API and dashboard as separate Render services with PostgreSQL (Neon) in production and SQLite for local development.

EXPERIENCE

Python Full Stack Intern | QSpiders, Gurugram

- Built backend applications and REST APIs using Python, Django, and Django REST Framework.
- Designed and tested CRUD endpoints and authentication flows for full-stack applications.
- Worked with MySQL and SQLite, including schema design and query debugging.
- Used Git, GitHub, and Postman for version control and API testing throughout development.

EDUCATION

B.Tech, Computer Science Engineering (AI & Machine Learning), 2021 - 2025
DPG Institute of Technology & Management, Maharshi Dayanand University | CGPA: 7.3

TRAINING & CERTIFICATIONS

- Python Full Stack Internship, QSpiders
- Cybersecurity Training, Honeywell & ICT Academy
- Power BI Training, Web Jyoti
- Complete Generative AI Course with LangChain and Hugging Face, Udemy